

1. Qué incluye este proyecto

Este repo implementa una API reactiva completa con Spring Boot 2.7.x y Spring WebFlux sobre MongoDB (reactive-streams-driver).

Incluye dos estilos de endpoints:

- API v1: controladores anotados (@RestController)
- API v2: functional routing (RouterFunction + Handler)

Además agrega:

- Manejo global de errores con Exceptions tipadas (NOT_FOUND, BAD_REQUEST, CONFLICT) y respuesta JSON.
- OpenAPI/Swagger UI (springdoc-openapi-webflux-ui) para documentación interactiva.
- Validación consistente (JSR-380) tanto en controllers como en functional routes.
- Observabilidad: Correlation-ID end-to-end + logs JSON con MDC + Actuator/Prometheus + tags custom.
- Tests con WebClient y Testcontainers (MongoDB) en JDK 11.

2. Endpoints importantes

Swagger UI:

- /swagger-ui.html

OpenAPI:

- /v3/api-docs

Actuator:

- /actuator/health
- /actuator/prometheus

API v1 (anotada):

- /api/v1/productos

API v2 (functional):

- /api/v2/productos

3. Cómo funciona WebFlux en este proyecto

WebFlux se basa en un modelo no-bloqueante:

- El servidor usa Netty (reactor-netty) y procesa requests con backpressure.
- Los controladores/handlers devuelven Mono (0..1) y Flux (0..N).
- Spring Data Reactive Mongo entrega publishers no bloqueantes.

Flujo típico (create):

HTTP POST -> Controller/Handler -> Validación -> Service -> ReactiveMongoRepository -> MongoDB -> Resp

Streaming (SSE):

GET /api/v1/productos/stream -> Flux<ProductoResponse> con delay -> text/event-stream

4. Manejo de errores: exceptions tipadas + handler global

Se usan exceptions tipadas:

- NotFoundException -> 404
- BadRequestException -> 400
- ConflictException -> 409

Todas se resuelven en GlobalErrorWebExceptionHandler (ErrorWebExceptionHandler), devolviendo un JSON consistente:

```
{
  "timestamp": 170..., "status": 400, "error": "Bad Request",
  "code": "BAD_REQUEST", "message": "...",
  "path": "/api/v2/productos", "correlationId": "...",
  "fieldErrors": [...]
}
```

En dev/test puede incluir más detalle (toggle `app.errors.includeStacktrace`).

5. Validación consistente (controllers + functional routes)

Controllers (API v1):

- Se usa `@Valid` sobre el request body.
- Bean Validation genera errores que terminan en el handler global.

Functional routes (API v2):

- Se usa `javax.validation.Validator` explícitamente.
 - `ValidationSupport.validateOrThrow(...)` produce `BadRequestException` con un mensaje normalizado.
- Esto evita mezclar `BeanPropertyBindingResult` con otras variantes y mantiene el mismo contrato de error.

6. Observabilidad: Correlation-ID + logs JSON + métricas

Correlation ID:

- `WebFilter` lee `X-Correlation-Id` o genera uno.
- Lo devuelve en el response header.
- Lo guarda en `Reactor Context` y `MDC` para logs.

`ReactorMdc`:

- Hook (`Hooks.onEachOperator`) propaga `correlationId` a través de operadores `Reactor` para que el `MDC` exista aun con cambios de thread.

Logs JSON:

- `logback-spring.xml` usa `logstash` encoder y agrega campos `app/env`.
- `correlationId` via `MDC` aparece en cada log.

Métricas:

- Actuator expone `/actuator/prometheus`.
- Se agregan tags comunes: `application` y `env`.

7. Cómo ejecutar

Local (dev):

- 1) `docker run --rm -p 27017:27017 --name mongo mongo:6`
- 2) `mvn spring-boot:run`

Docker Compose:

- `docker-compose up -d --build`

Luego:

- `http://localhost:8080/swagger-ui.html`
- `http://localhost:8080/actuator/health`
- `http://localhost:8080/actuator/prometheus`

8. Tests (JDK 11) con Testcontainers MongoDB

- `ProductoRepositoryTest`: `@DataMongoTest` + `MongoDBContainer` + `DynamicPropertySource`
- `ProductoControllerTest` / `FunctionalControllerTest`:
`@SpringBootTest(RANDOM_PORT)` + `WebTestClient` + `MongoDBContainer`

Ejecutar:

- `mvn test`